Assignment 1: Observability Report

Enterprise Software Development, Fall 2026

Part A: Project

Problem Statement

The problem this project addresses is the difficulty of booking movie seats online without knowing whether a seat is available in real time. When multiple users try to book the same seat at the same time, systems without proper hold logic end up with double bookings or race conditions.

Intended Users

The intended users are moviegoers who want to browse showtimes, hold seats temporarily while they decide, confirm a booking, and cancel if needed.

Solution

The solution is a REST API built with FastAPI and Python. It uses a SQLite database to store movies, showtimes, seats, and bookings. Seats can be placed in a "held" state for 120 seconds, after which they are automatically released by a background task. Users then confirm their booking, which triggers a simulated payment step.

How to Run

All services are started with a single command:

```
docker compose up --build -d
```

This brings up 7 containers: the FastAPI app, Prometheus, Grafana, Node Exporter, Elasticsearch, Kibana, and Filebeat. The API is available at http://localhost:8000 and the interactive docs at http://localhost:8000/docs.

To test a booking flow:

```
POST /showtimes/1/hold  --  hold seats
POST /bookings/confirm  --  confirm the booking
POST /bookings/{id}/cancel  --  cancel if needed
```

Part B: Metrics

Overview

Prometheus and Grafana are used to collect and visualise both application-level and system-level metrics. All four Prometheus metric types are used across the project.

Metrics Table

Metric Name	Type	Unit	Purpose	Labels
bookings_total	Counter	bookings	Total confirmed bookings	showtime_id
booking_failures_total	Counter	failures	Failed booking attempts	reason
cancellations_total	Counter	cancellations	Total cancellations	showtime_id
seats_held_current	Gauge	seats	Seats currently on hold	none
http_request_duration_seconds	Histogram	seconds	Request latency per endpoint	endpoint, method
payment_duration_seconds_histogram	Histogram	seconds	Payment step duration	none
payment_duration_seconds_summary	Summary	seconds	Average payment duration	none
demo_requests_total	Counter	requests	Cardinality demo counter	request_id (demo only)

Grafana Charts and Queries



Bookings confirmed (rate/min)

```
rate(bookings_total[1m]) * 60
```

This chart shows how many bookings are being confirmed per minute. It helps spot unusual spikes or drops in booking activity.

Seats currently held (Gauge)

```
seats_held_current
```

This shows the live count of seats in the "held" state. It goes up when a hold is placed and down when it expires or is confirmed.

HTTP request duration p95 by endpoint (Histogram)

```
histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[1m]))
```

This shows the 95th percentile response time for each endpoint. During the fault experiment, the /movies endpoint spiked to around 1 second while other endpoints stayed near zero.

Payment duration average (Summary)

```
payment_duration_seconds_summary_sum /
payment_duration_seconds_summary_count
```

This gives the rolling average time spent in the payment step. Python summaries do not support percentiles, so a histogram is used alongside it for p95.

Node Exporter: CPU and Memory

```
100 - (avg
by(instance)(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

```
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
```

These charts show CPU usage percentage and memory usage fraction for the host machine (movie-booking-host). They are collected by Node Exporter and scraped by Prometheus.

Part C: Logs

What is Logged and Why

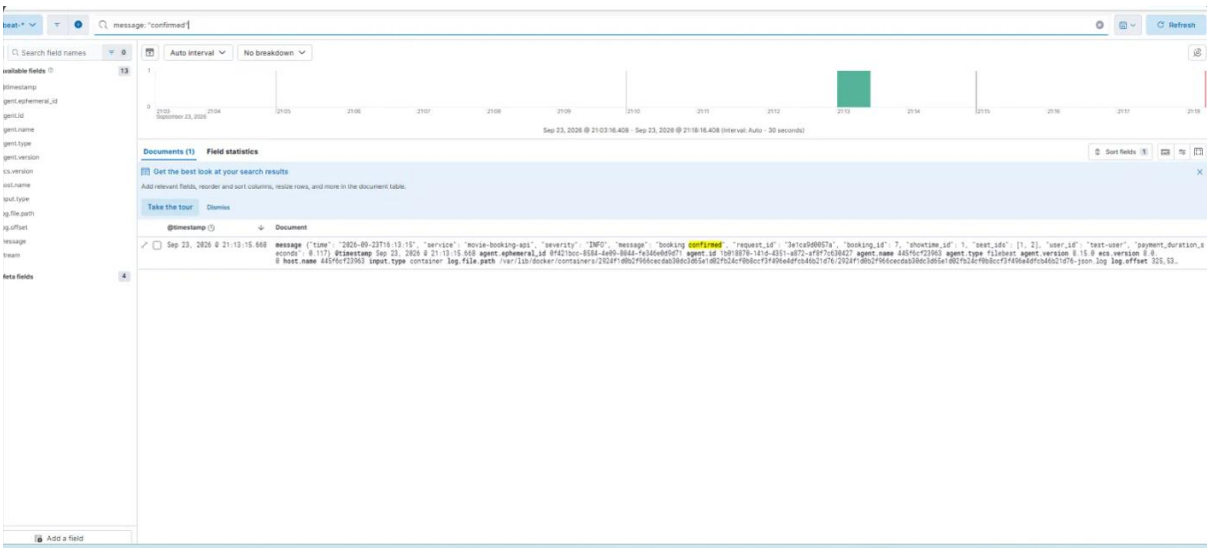
The application logs structured JSON lines to stdout for every significant event. This includes:

- Seats being held or expired, so we can trace booking flow issues
- Bookings being confirmed or cancelled, with booking ID and request ID for tracing
- Warnings when a hold has expired before confirmation
- Errors when a payment fails
- App startup events

Every log line includes: time, service name, severity (INFO/WARNING/ERROR), message, and request_id. Business events also include booking_id, showtime_id, seat_ids, and user_id.

Example Log Entry (original)

```
{"time": "2026-09-24T03:47:59", "service": "movie-booking-api",
"severity": "ERROR", "message": "database connection timeout",
"request_id": "862a044cdafc", "endpoint": "/movies"}
```



How Filebeat Collects Logs

Filebeat uses the container input type, which reads Docker container log files from /var/lib/docker/containers/*/*.log. Each line in these files is a JSON object written by Dockerss json-file logging driver. Filebeat is configured with the ndjson parser, which unwraps the Docker envelope and promotes all fields (time, service, severity, message, request_id) to top-level searchable fields in Elasticsearch.

Where Logs Live

Logs are stored in Elasticsearch in a named Docker volume (es_data). This means they survive container restarts. Docker also keeps the raw log files on disk at /var/lib/docker/containers/ until the container is removed. Old logs are not automatically deleted unless log rotation is configured, which was not set up for this project.

How to Search in Kibana

To find a specific error, go to http://localhost:5601/app/discover, select the filebeat-* data view, and use KQL in the search bar. For example:

```
message: "database connection timeout"
```

To find a specific request:

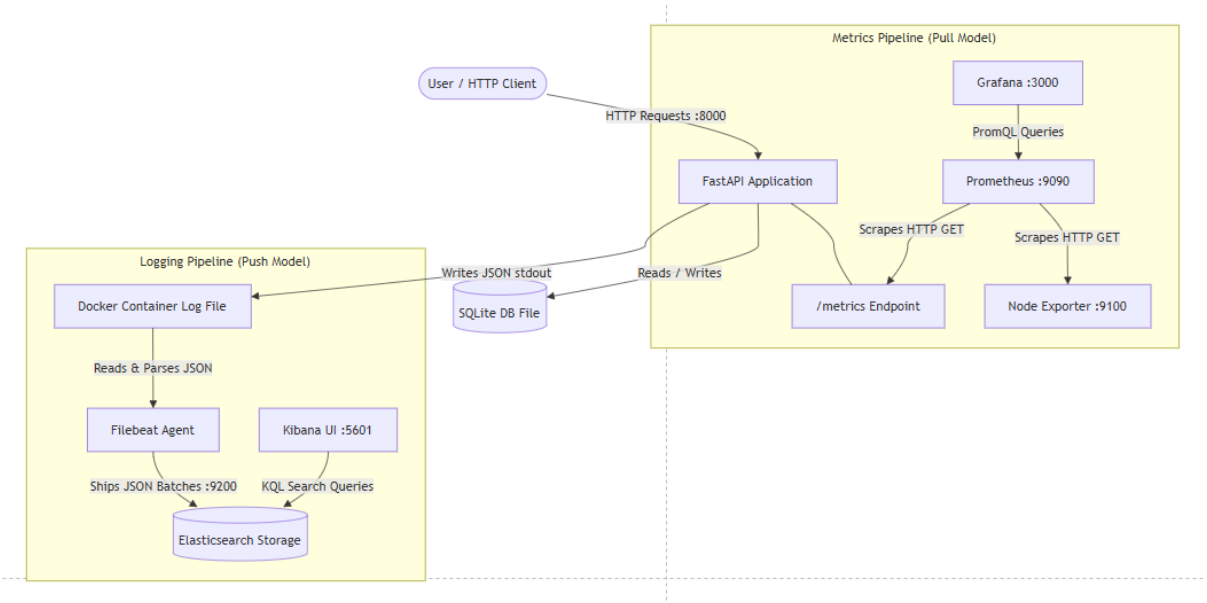
```
request_id: "862a044cdafc"
```

To filter by severity:

```
severity: "ERROR"
```

Part D: System Design

1. Architecture Diagram



The diagram shows all components and how they connect. The FastAPI app sits at the centre. It handles user requests, reads and writes to the SQLite database, exposes a /metrics endpoint for Prometheus, and outputs structured JSON logs to stdout. Everything else is observability tooling built around it.

There are two separate pipelines: one for metrics and one for logs. They run independently, so if one breaks the other keeps working, and neither one affects the actual booking app.

2. How the Components Work Together

Metrics pipeline (pull model)

Prometheus works by pulling. It sends an HTTP GET to /metrics on the FastAPI app every 15 seconds and records whatever it finds. It does the same to Node Exporter to collect CPU, memory, disk, and network stats from the host machine. All this data is stored as time series in Prometheus. Grafana then connects to Prometheus as a data source and uses PromQL queries to turn that data into dashboard charts.

Logging pipeline (push model)

Logs work the opposite way. FastAPI writes every log event as a JSON line to stdout. Docker captures this and saves it to a log file under /var/lib/docker/containers/. Filebeat watches that folder, reads new lines as they appear, parses the JSON fields, and ships batches to Elasticsearch at port 9200. Elasticsearch indexes and stores the logs, and Kibana connects to it so we can search them using KQL.

3. Where Data Lives and Why

Data	Where it lives	Why
Bookings, seats, movies	SQLite file on disk	Simple, no extra service needed for a small app
Metrics (time series)	Prometheus volume	Fast for time-range queries, built for this purpose
Logs	Elasticsearch volume	Built for full-text search across many fields
Dashboard configs	Grafana volume	Persists dashboard settings across restarts

4. What Happens if a Component Fails

- Prometheus goes down: Grafana shows no data, but the app keeps running normally. No metrics are collected during the outage but no user requests are affected.
- Elasticsearch or Filebeat goes down: logs stop being shipped and searchable, but the app still writes logs to stdout and Docker still saves them to disk. When Filebeat recovers it picks up from where it left off using its registry file.
- Grafana or Kibana goes down: we lose visibility but nothing breaks for users.
- The FastAPI app goes down: everything else keeps running but there is nothing to observe.

The key design decision is that observability runs completely out of band. Metrics scraping and log shipping happen asynchronously and never block a user request. If Filebeat is slow or Prometheus misses a scrape, the booking endpoint still responds normally.

5. Following a Metric: http_request_duration_seconds

- In the code: the middleware in main.py records the time before and after every request and calls .observe(duration) on the histogram metric.
- Prometheus collects it: every 15 seconds Prometheus scrapes /metrics, reads the histogram buckets, and updates its time series database.
- Grafana displays it: the dashboard uses histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[1m])) to calculate p95 latency per endpoint and plots it as a line chart.

During the E.1 fault experiment this metric clearly showed the /movies endpoint jumping to around 1 second p95 while all other endpoints stayed near zero.

6. Following a Log: booking confirmed

- In the code: after a successful booking, main.py calls log("INFO", "booking confirmed", booking_id=..., request_id=...) which formats a JSON line and writes it to stdout.
- Docker saves it: Dockers's json-file driver wraps the line and appends it to the container log file on disk.
- Filebeat reads and parses it: using the ndjson parser in filebeat.yml, Filebeat extracts all JSON fields as top-level searchable fields.
- Elasticsearch stores it: the event is indexed under filebeat-8.15.0 with a timestamp.

- Kibana finds it: searching message: "booking confirmed" in Discover returns the exact log entry with all fields visible.

Part E: Experiments

E.1: Reproducing a Problem

The Problem

The fault introduced was a simulated database timeout on the /movies endpoint. Every request to /movies was given an artificial 0.6 second delay and then returned a 500 error with the message "Database connection timed out". This simulates what happens when the database becomes unresponsive under load.

Normal Behaviour (Baseline)

Before the fault was introduced, the HTTP request duration p95 chart in Grafana showed all endpoints responding in under 0.01 seconds. No ERROR logs appeared in Kibana for the /movies endpoint.

Prediction

The following changes were expected when the fault was introduced:

- The p95 latency for /movies in Grafana would spike to around 0.6 to 1 second
- ERROR level logs with the message "database connection timeout" would appear in Kibana
- All requests to /movies would return 500, captured as exceptions in the traffic loop

Introducing the Fault

The /movies endpoint was modified to include a time.sleep(0.6) call and a raised HTTPException with status 500. The app was rebuilt and 15 requests were sent:

```
1..15 | ForEach-Object { Invoke-WebRequest -Uri
"http://localhost:8000/movies" ... }
```

What Actually Changed

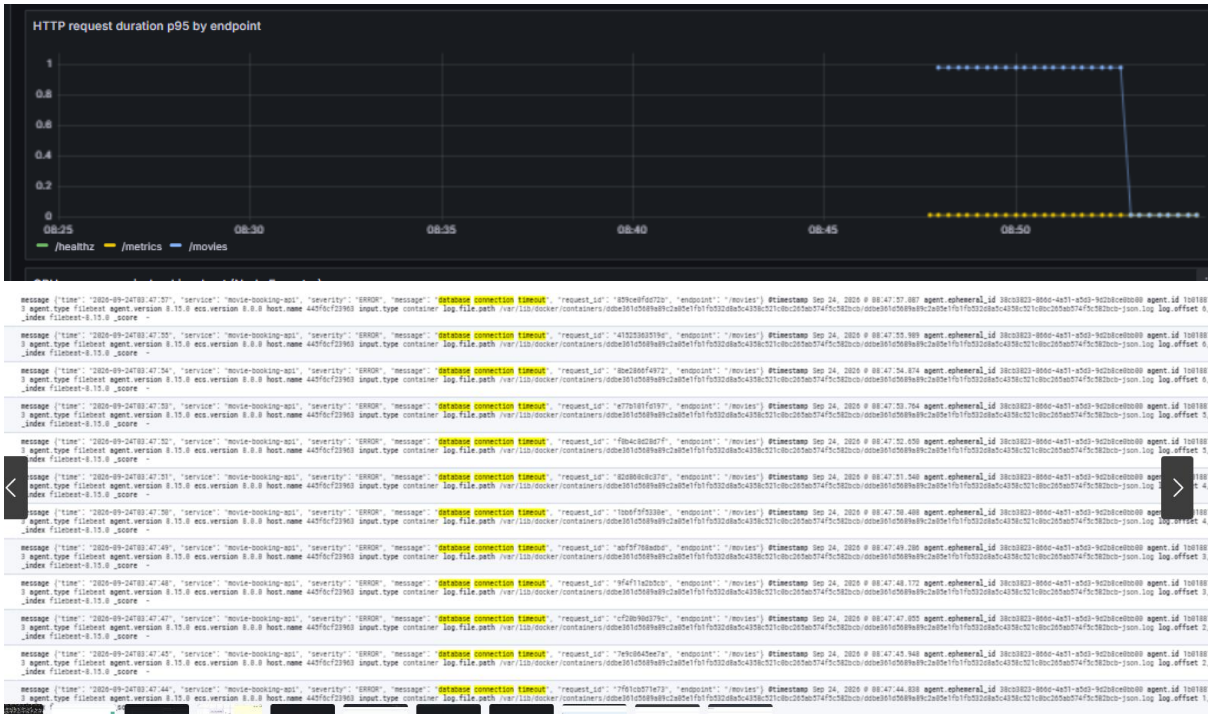




- Grafana: the /movies line on the HTTP request duration p95 chart jumped to 1 second, clearly visible as a spike while /healthz and /metrics stayed flat near zero.
- Kibana: 38 ERROR documents appeared with message "database connection timeout", each with a unique request_id and the endpoint field set to /movies.
- Users: every request to /movies returned a 500 error. The rest of the app (holding seats, confirming bookings) continued to work normally.

Recovery

The fault was removed by reverting the /movies endpoint to its original implementation. The app was rebuilt and the traffic loop was run again. In Grafana, the p95 latency for /movies dropped back to near zero. In Kibana, no new ERROR logs appeared after the recovery point, confirmed by the document count dropping from 38 to 15 visible in the current time window.



E.2: Cardinality Explosion

What is Cardinality Explosion

Cardinality explosion happens when a Prometheus metric label has many different values, causing Prometheus to create a separate time series for each unique combination. Labels like request IDs or user IDs are the most common cause because every request creates a brand new label value, meaning a brand new time series in Prometheus memory.

The Demo

The /demo/cardinality-bad endpoint was used to demonstrate this. Each call to that endpoint creates a Counter increment with a unique request_id label value. The endpoint was called 200 times:

```
1..100 | ForEach-Object { Invoke-WebRequest -Uri
"http://localhost:8000/demo/cardinality-bad" -Method POST ... }
```

After the calls, the following query was run in Prometheus:

```
count(demo_requests_total)
```

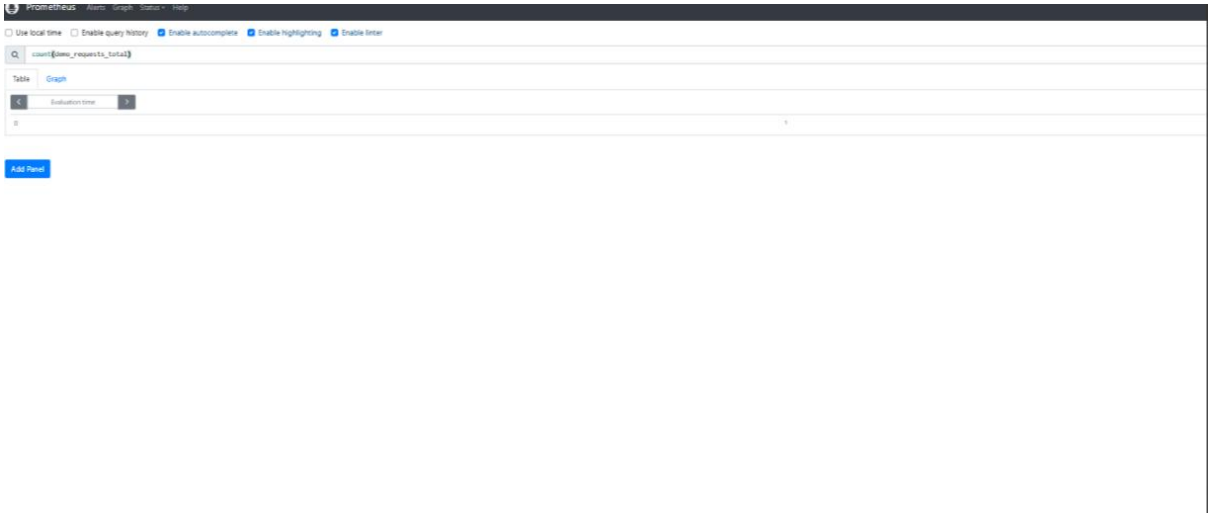
Result: 200 unique time series, one per request_id value.



Removing the Label

The request_id label was removed from the DEMO_REQUESTS_TOTAL metric definition in metrics.py. The counter was changed from a labelled counter to a plain counter with no labels. The app was rebuilt, a few more calls were made, and the same Prometheus query was run again.

Result: count(demo_requests_total) = 1. Only a single time series exists regardless of how many requests are made.



Why This Matters at Scale

At larger scale, cardinality explosion is a serious problem. Each time series consumes memory in Prometheus. With 200 series from 200 requests, the impact is small. But in a production system handling thousands of requests per minute, using request_id as a label would create millions of time series within hours, causing Prometheus to run out of memory or become too slow to query. This is why request IDs and user IDs belong in logs, not in metric labels. Logs are built for high cardinality individual events, while metrics are designed for low cardinality aggregated data.